University of Rwanda
College of Science and Technology
Sep school of ICT
Department of computer and software Engineering
Year 3

Module Title : Mobile Application Systems and Design.

Reg No: 223007592
Date: 25th March, 2026
INDIVIDUAL Flutter lab 5: Posts Manager with SQLite.

Qn 1.

(i) Dependencies used and why.

- Sqflite: ^2.3.0 : This provides the API to create, open, and querry a local SQlite database on Android and iOS.
- Sqflite_common_ffi_web: ^1.1.1 : This bridges same database code to run in the browser without changes since sqflite alone only supports mobile.
- Path_provider ^2.1.0 : Provides the correct local storage directory for each platform.
- intl: ^0.20.2 : This was used for formatting datetime objects into readable strings like "March 25, 2026" in the UI. Without it, dates would display as raw Iso strings
- google_fonts: ^8.0.2: This provides the poppins font used throughout the app for clean typography.
- flutter_staggered_animations: ^1.1.1 : This adds staggered fade and slide animations to the post list making the UI feel polished and responsive when posts load.

(ii) SQlite is necessary for local storage because;

- No internet required, since it is a file-based database stored directly on the device.
- Data persists between sessions. Unlike regular variables, SQlite data survives app restarts and app updates; a post created today will be there a month later.
- It supports full SQL queries, making it easy to filter, sort, and search posts by title or content unlike Isolo.
- It is lightweight, built directly into Android and ios, and requires no server, authentication, or extra cost.

Qn 2. Handling database exceptions.

(i). Handle database not initialized: In "database_helper.dart", the "initDatabase()" method is wrapped in a try-catch. If the database fails to open or the table fails to create, a descriptive exception is thrown immediately. On web, "main.dart" sets the correct factory before any database call.

(ii) Handling insert, update and delete errors.

Every CRUD method in DatabaseHelper wraps its operation in a try-catch and rethrows a human-readable exception. The UI layer then catches this and shows it in a snackbar instead of crashing.

Also, conflictAlgorithm.abort ensures that if a conflicting insert is attempted, it throws immediately rather than silently overwriting data.

(iii) Handling invalid or corrupted data.

In post_dart, the fromMap() factory uses null-safe casting on every field. If a database row is missing a field due to corruption or a schema mismatch, the app falls back to a safe default instead of crashing with a null cast error.

Additionally, constructor-level assertions also prevent invalid post objects from ever being created in the first place.

Qn 3. SQLite in Flutter

(i) Database vs Table

A database is the container - a single .db file stored on the device for instance posts_manager.db whereas a table is a structured grid of rows and columns inside the database, for instance, "posts" has columns for id, title, content, author, created_at and updated_at.

(ii) CRUD operations

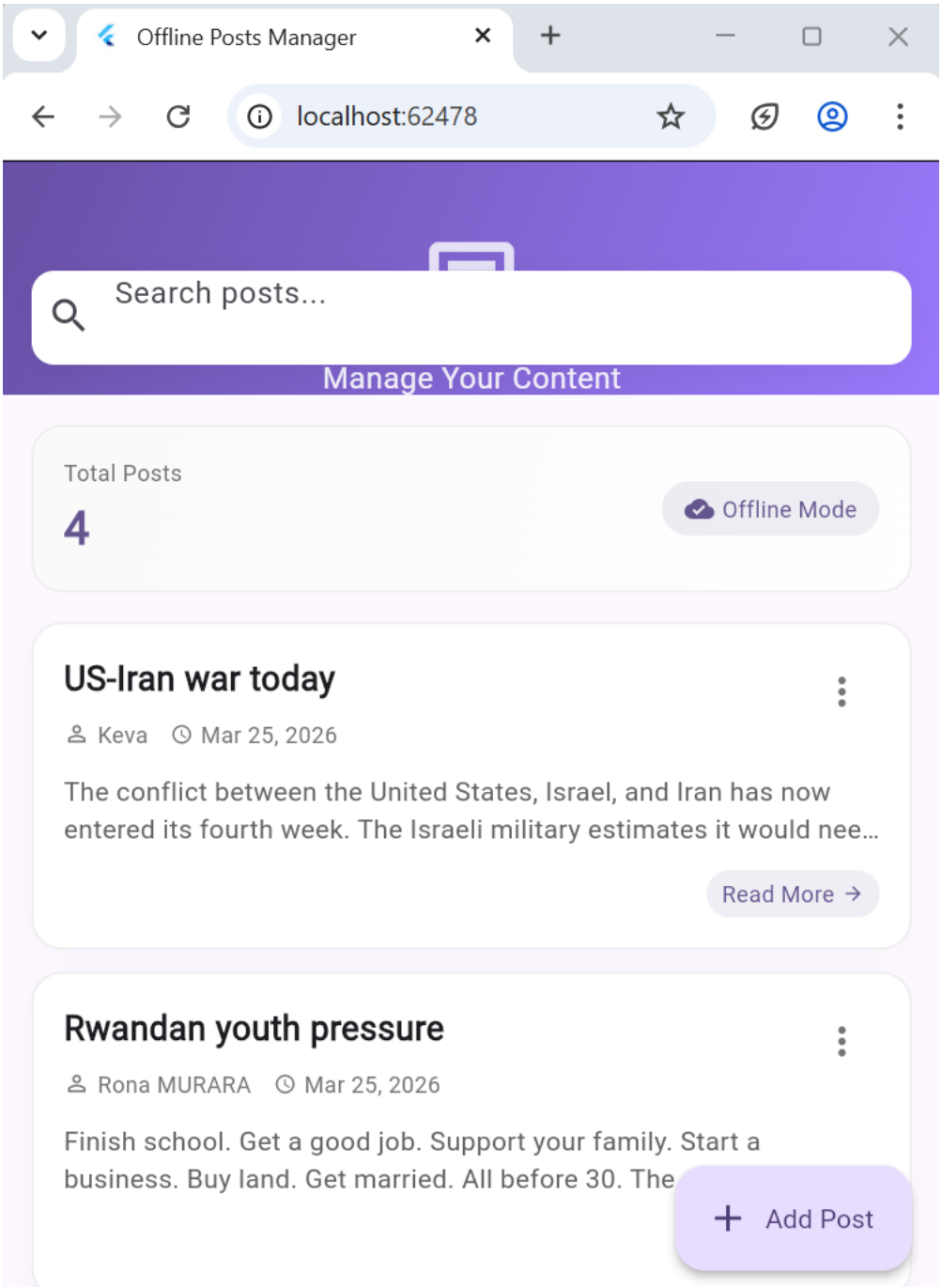
Each method receives or returns a post object. The toMap() method converts a post into a Map<String, dynamic> that SQLite understands, and fromMap() converts a raw database row back into a post object.

(iii) How Flutter interacts with the database asynchronously.

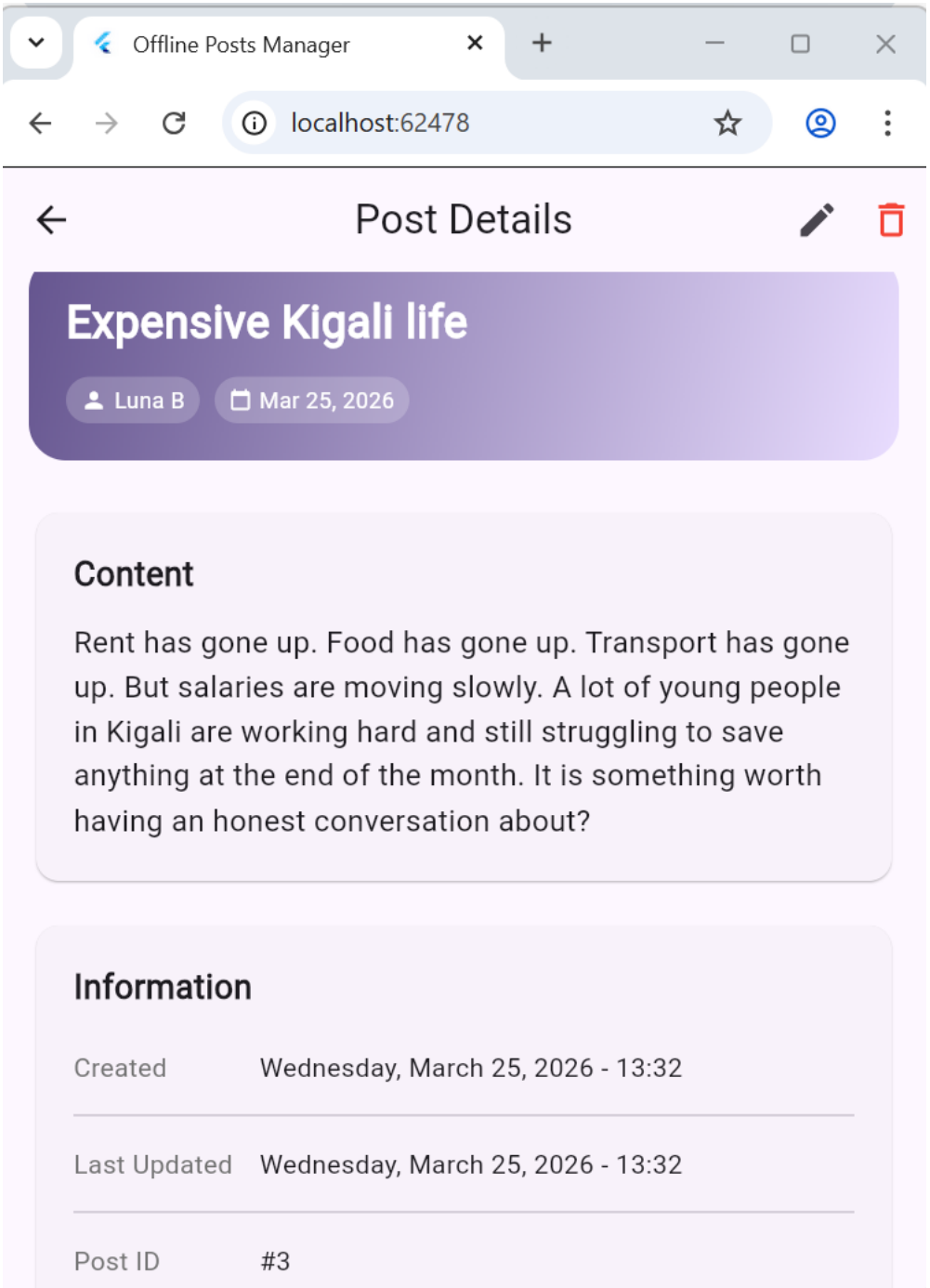
SQLite operations involve reading and writing to disk, which takes time. Flutter uses "async/await" to handle this without freezing the UI. Every database method is marked "async" and returns a "Future". The UI calls these methods with "await" inside its own async functions, and uses "setState()" with an "_isLoading" flag to show a loading spinner while waiting.

SCREENSHOTS.

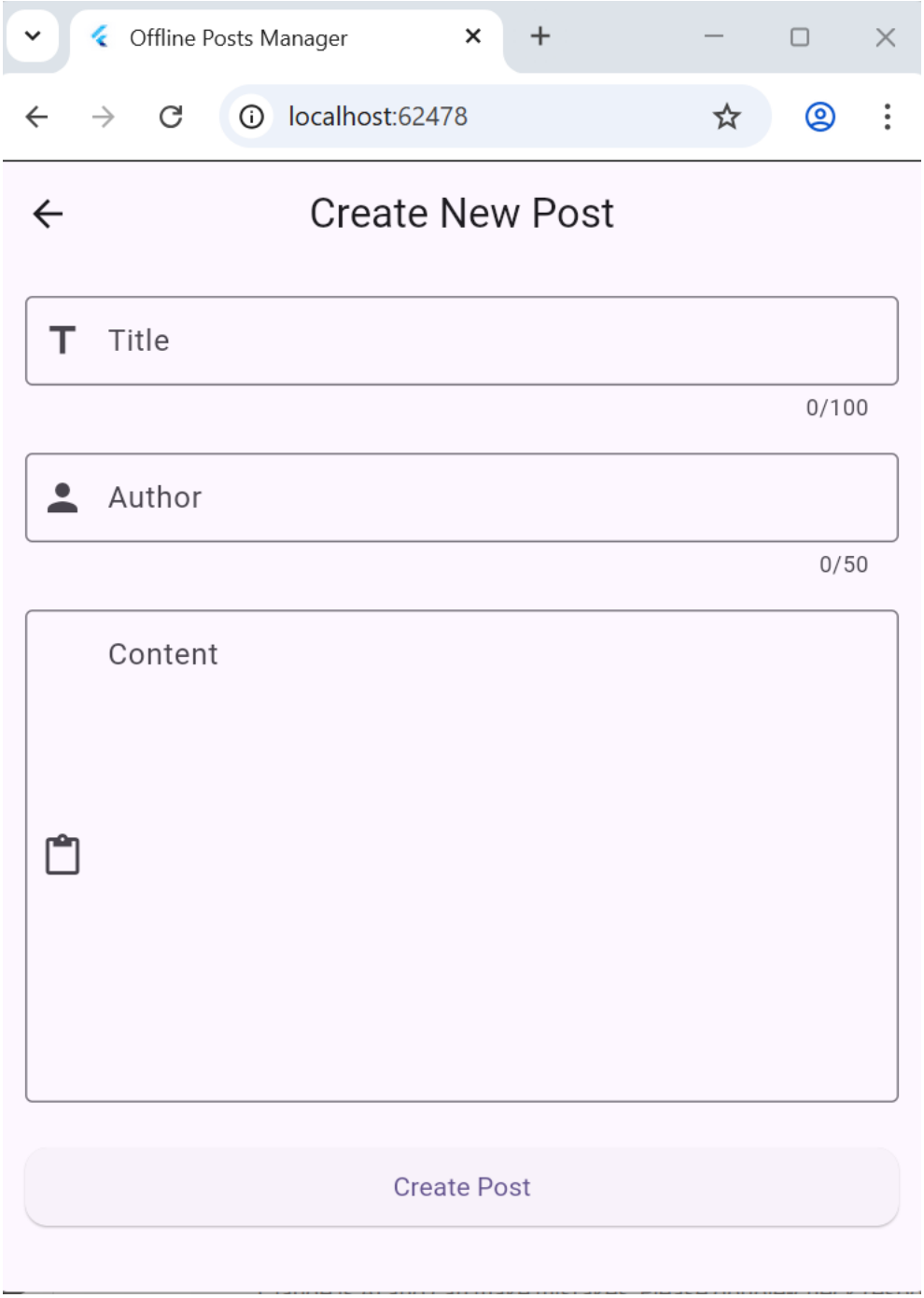
1. **Viewing all posts stored locally.**



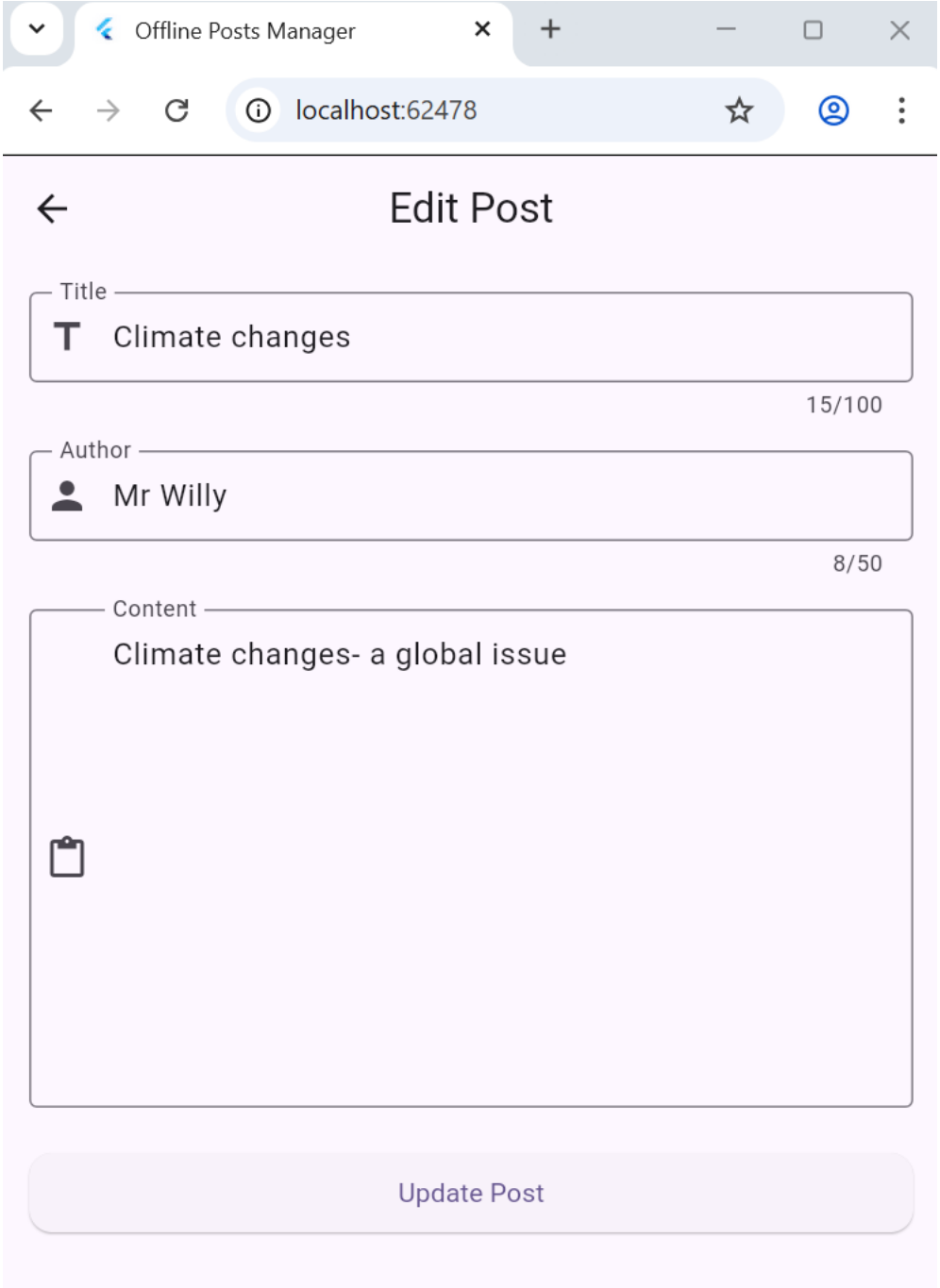
2. **Reading details of a post.**



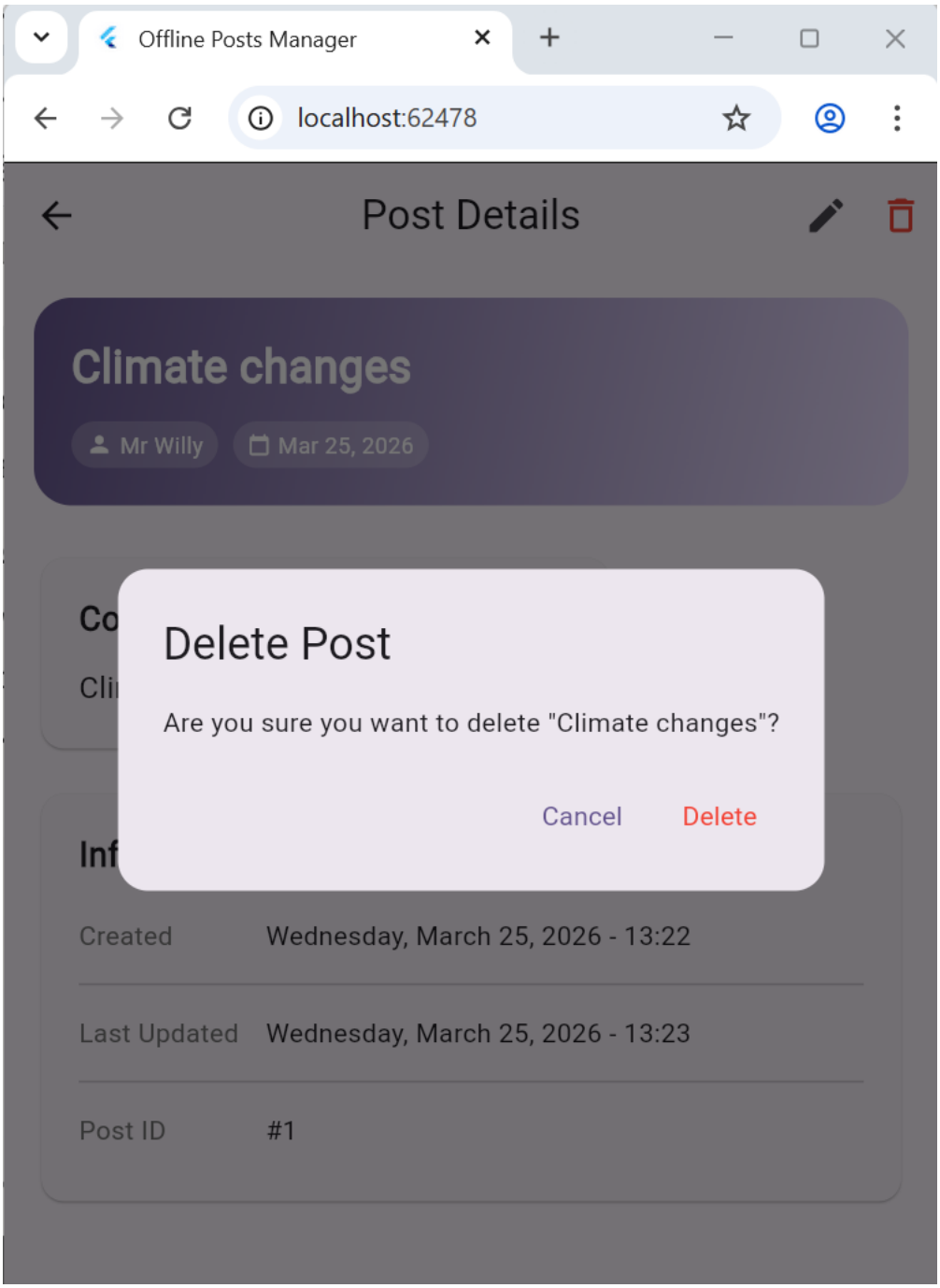
3. **Adding a new post.**



4. **Editing an existing post.**

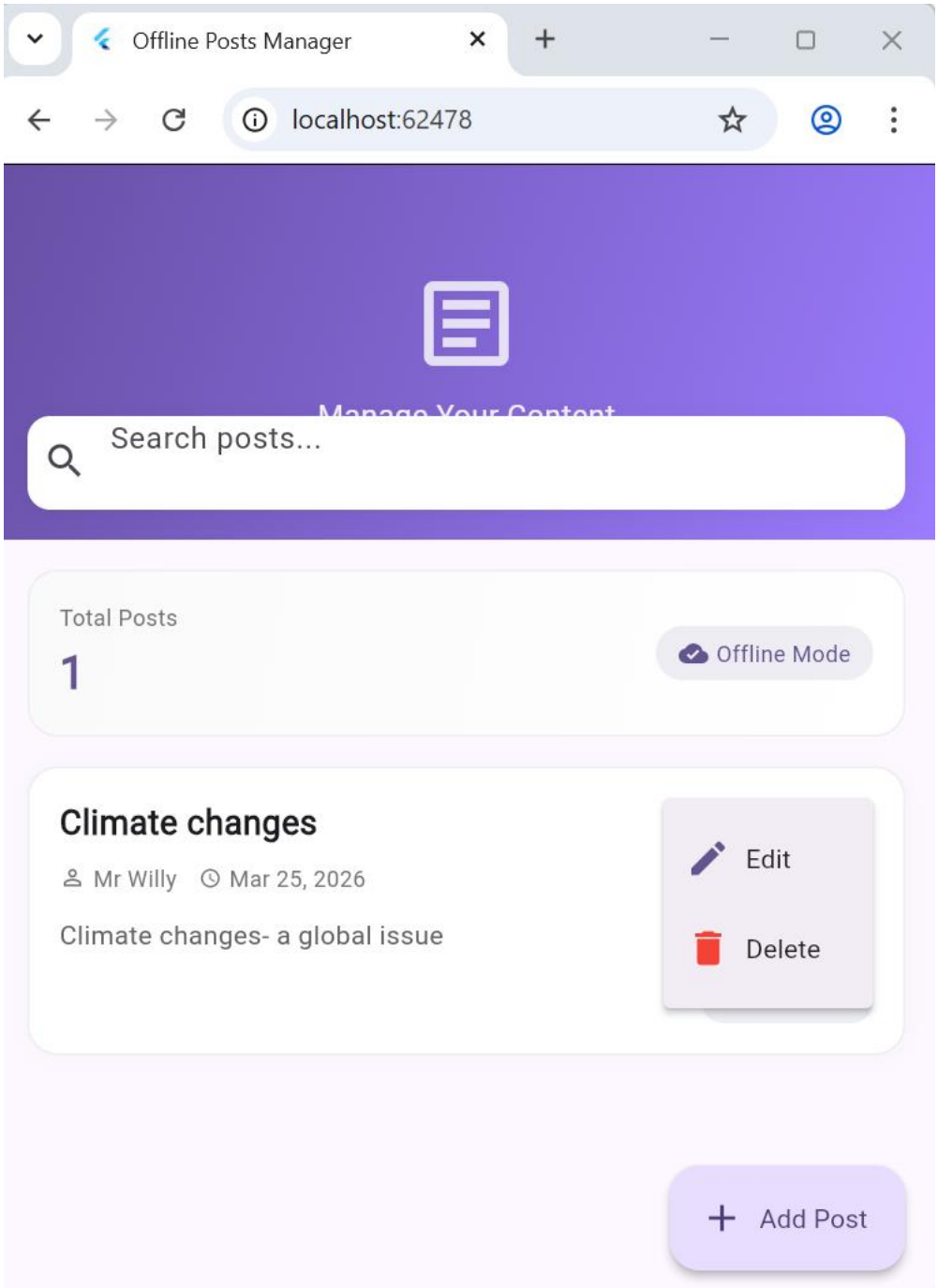


5. **Deleting a post.**

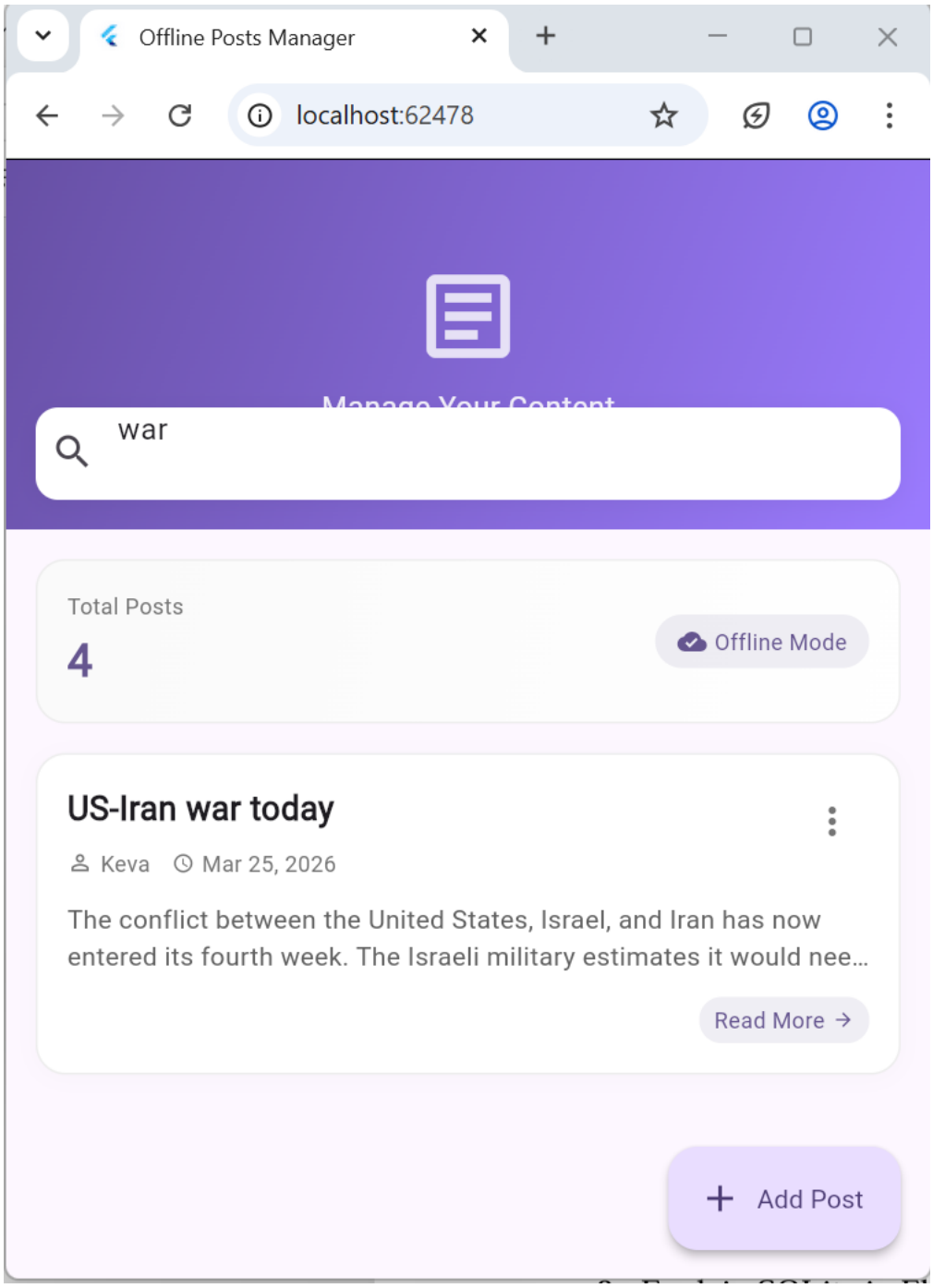
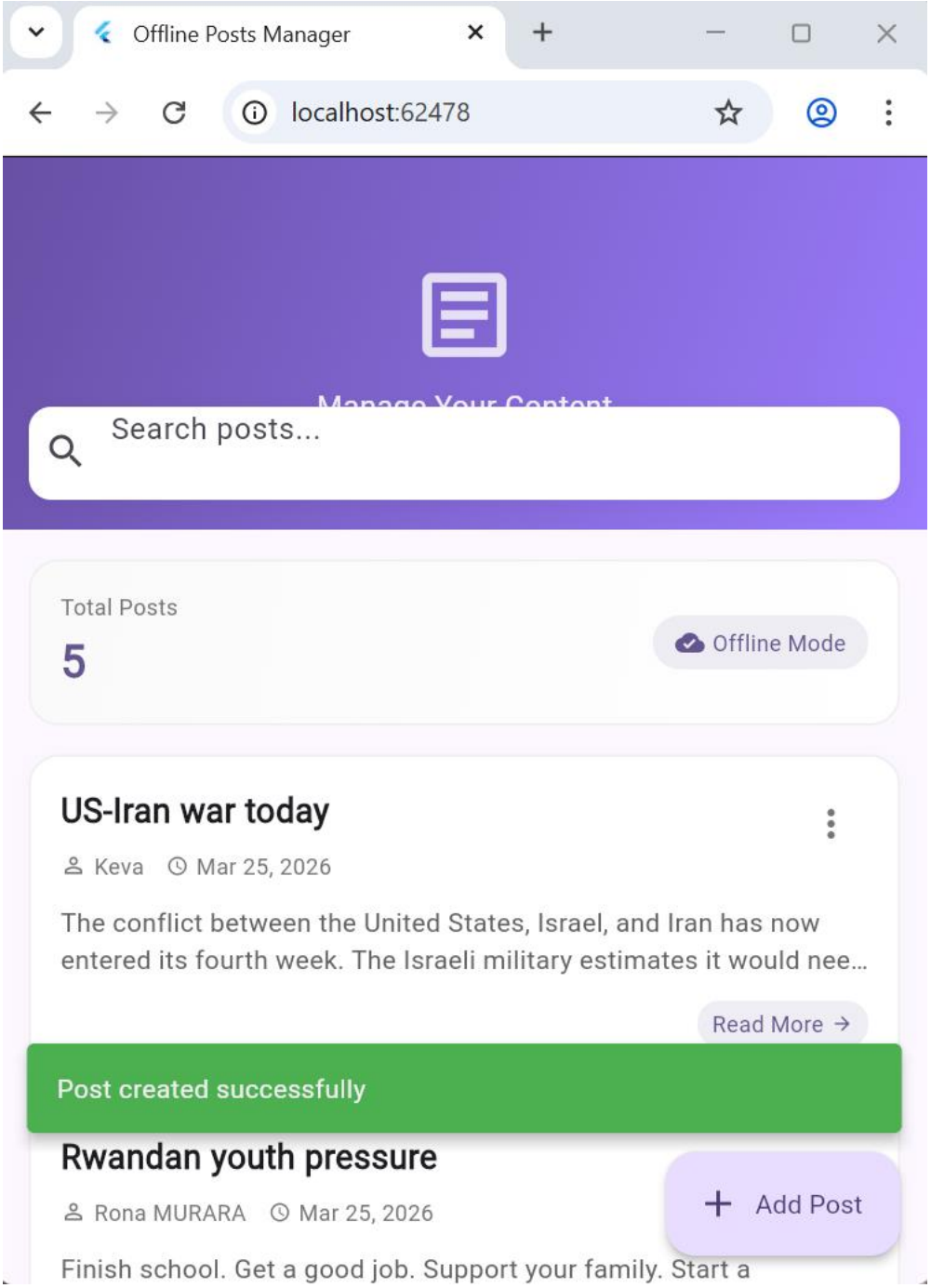


Additional screenshots

i. Post menu (For editing or deleting).



ii. Searching post.



iii. Snack bar notification.